ESE 224 Stock Sim Project Report:
By Rubayeth Hasan and Maksym Ryvak

Introduction:

StockSim is a historical market analyzer and trading strategy simulator that is written in C++. The objective of this project was to combine several of the data structures we have reviewed in ESE 224 into one practical system that can load real historical stock market data, store it efficiently, and use it to test different investment strategies. Instead of treating linked lists, queues, stacks, trees, vectors, templates, and object oriented programming as separate topics, this project aims to modularize and integrate them inside one working command line application.

The main problem being studied is whether a Dynamic SIP strategy, which invests more aggressively during market dips, can outperform a Fixed SIP strategy over a long historical period. SIP stands for systematic investment plan. A Fixed SIP invests the same dollar amount at regular intervals, while a Dynamic SIP changes the timing or amount of investment based on market behavior. The project asks whether reacting to price drops can improve performance compared with simply investing a fixed amount every month.

The dataset consists of Yahoo Finance-style CSV files containing historical price data. Each row stores the following: a trading date, open price, high price, low price, close price, adjusted close price, and trading volume. The required analysis period is from January 1, 2000 through January 1, 2020. This is an important testing window because it includes and allows us to view market environments during several conditions: the dot-com crash, the 2008 financial crisis, and the post-2010 bull market. Because the stock market is closed on weekends and holidays, the program cannot assume that every linked-list node represents one calendar day. Instead, each node represents one actual trading day that we obtain from the integrated CSV files.

The motivation for this project is that market simulation is a realistic application of data structures. A linked list can store sequential price history. A stack can store trade history for undo operations. A queue can store pending orders. A circular queue can track moving averages. A binary search tree can organize stocks or strategies by return. Templates allow the program to manage different asset types generically. The result is a project that demonstrates both programming skill and financial analysis.

AI Usage Log:
Prompt:
Help me write an introduction for a C++ StockSim project that explains the problem statement, motivation, dataset, and research question.

AI Output:
AI provided a general introduction explaining the project purpose and data structure motivation.

Modification:
We edited the wording to better match our specific report and project implementation.

Data Structure Design:

PriceHistory Doubly Linked List:

The PriceHistory class stores historical market data using a doubly linked list. Each node is a PriceNode that contains the date, open price, high price, low price, close price, volume, and pointers to both the next and previous nodes. We chose a doubly linked list because stock data is sequential by nature, each trading day follows the previous trading day, and many strategy calculations require walking forward or backward through time.

The linked list appends new price records as the CSV file is read. This is the efficiency of using a DLL, because the new node can be attached directly to the tail. This means the time complexity of append() is O(1). Searching for a specific date with findByDate() requires scanning from the head until the date is found, so the time complexity is O(n). Printing a date range is also O(n) because the program may need to inspect each node to determine whether it falls within the requested range.

The forward iterator begins at the head and moves using the next pointer. This is useful for displaying price history and for strategy backtesting, where the simulation moves from older dates to newer dates. The reverse iterator begins at the tail and moves using the prev pointer. This is especially useful for strategies like six-month momentum, where the program may need to examine previous prices from a current point in time.

A doubly linked list preserves chronological order while allowing both forward and backward traversal. It also avoids using prohibited STL containers such as std::list.

CircularQueue:

The CircularQueue class is used to store a fixed number of recent closing prices. Its main purpose is calculating moving averages, such as the 50-day and 200-day moving averages used in the Golden Cross strategy.

The circular queue is implemented with a dynamically allocated array, along with head, tail, capacity, and count variables. When a new value is added and the queue is already full, the oldest value is overwritten. This behavior is ideal for moving averages because only the most recent N values are needed.

The time complexity of enqueue() is O(1) because the program only updates an array position and advances an index. dequeue() and peek() are also O(1). The getAverage() method is O(n) because it loops through the current queue values to compute the sum. For this project, that is acceptable because the moving average windows are fixed sizes, such as 50 and 200 days.

The circular queue is the right structure because it avoids shifting elements every time a new price is added. A normal array would require moving values around, but the circular queue reuses array space efficiently.

TradeStack:

The TradeStack class stores completed trades in LIFO order. Each TradeRecord stores the ticker, date, price, number of shares, action type, and total cost. The stack is implemented using linked nodes, where each new trade is pushed onto the top.

The main purpose of this stack is to support the undo feature in the portfolio. If a user buys or sells shares by mistake, the most recent trade can be removed and reversed. This matches stack behavior perfectly because the most recent action should be the first one undone. The time complexity of push() is O(1) because a new node is added at the top. pop() is also O(1) because only the top node is removed. peek() is O(1) because it only reads the top node. Printing all trades is O(n) because every stack node must be visited.

This stack is the right choice because undo operations naturally follow LIFO logic. The program should not undo an older trade before undoing the most recent trade.

OrderQueue:

The OrderQueue class stores pending trade orders in FIFO order. Each order contains the ticker, order type, side, target price, number of shares, and submission date. The queue is implemented using linked nodes with front and rear pointers.

The reason for using a queue is that pending orders should be handled in the order they were submitted. If a user enters multiple orders, the first order should be checked and executed before later ones. This matches FIFO behavior.

The time complexity of enqueue() is O(1) because the new order is added to the rear. dequeue() is O(1) because the front node is removed. peek() is O(1) because it only reads the front order. Printing all pending orders is O(n).

The queue is the right choice because it models real pending order behavior. It also takes into account the project requirement to manually implement a queue rather than using std::queue.

StockBST:

The StockBST class organizes stocks or strategy results using a numerical key, such as annual return, volatility, or final portfolio value. Each node stores a ticker, key, year, left pointer, and right pointer. Lower keys go to the left subtree, and higher keys go to the right subtree.

The BST allows the program to display results in sorted order. An inorder traversal prints the nodes from lowest key to highest key. This is useful for comparing annual returns or ranking Dynamic SIP parameter sweep results.

The average time complexity of insert() is O(log n) if the tree is reasonably balanced. However, in the worst case, if values are inserted in sorted order, the tree can become skewed and insertion becomes O(n). Searching has the same average and worst-case behavior. A range

search is O(n) in the worst case because it may need to visit many nodes, but it can skip unnecessary branches when the current node's key is outside the requested range. The traversal methods are all O(n) because every node is visited once. The height calculation is O(n) because it recursively checks all nodes.

The BST is appropriate because the project needs sorted financial results without using maps or hash tables. It gives a clear connection between tree structure and financial ranking.

Portfolio Vector:

The Portfolio class uses a std::vector to store current holdings. This is allowed by the project instructions for portfolio positions. Each position stores the ticker, shares, average cost basis, and current price.

A vector is a good choice for holdings because the number of portfolio positions is usually small compared with the number of price records. Searching for a ticker is O(n), but that is acceptable for a small portfolio. Sorting holdings by ticker or return uses std::sort, which is O(n log n).

The portfolio also integrates the custom stack and queue. The TradeStack stores completed trades, while the OrderQueue stores pending orders. This design keeps portfolio state, order execution, and undo history connected.

AI Usage Log:

Prompt:

Provided is my current integrations for the data structure aspect of my stocksim project. I want you to break down each individual data structure class into a summary and description of what it is doing.

AI Output:

The AI output a detailed summary of each data structure class.

Modification:

Some of the details were lengthy and not all that necessary, so I did make the descriptions shorter and more targeted.

Trading Strategies:

1. Fixed SIP Strategy

The Fixed SIP strategy invests the same dollar amount on the first available trading day of each month. For example, if the monthly capital is $500, then the strategy purchases $500 worth of the asset every month using the closing price on that trading day. Fractional shares are allowed in the simulation so the entire monthly amount can be invested. This strategy is simple

and consistent. It does not try to predict the market or react to price movement. Its expected behavior is steady accumulation over time. It should perform well during long-term upward markets because it continuously buys shares regardless of short-term volatility. However, it may miss opportunities to invest more during major market dips.

Parameters Used:

- Monthly capital
- Start year
- End year
- Asset price history

Expected behavior:

- Stable number of trades
- Fully budget-neutral
- No market timing
- Useful baseline for comparison

2. Dynamic SIP Strategy

The Dynamic SIP strategy combines a macro trend filter with a mean-reversion panic override. By default, it uses a 200-day moving average to detect market regimes: it hoards the $500 monthly capital in cash during bear markets (price < 200MA) and deploys it during bull markets (price > 200MA). However, if the market crashes so violently that the current price stretches more than 32.5% below the 200-day MA, it triggers a "rubber band" override. It immediately begins buying the panic dip, draining the accumulated cash hoard aggressively at a 70% monthly rate to maximize share accumulation at the generational bottom.

Parameters Used:

- Monthly capital = 500
- Start year = 2000
- End year = 2020
- dipThreshold = 32.5 (Panic Override: % below 200-day MA)
- rallyThreshold = 0.0 (Neutralized)
- multiplier = 0.70 (Hoard Drain Rate)

Final Portfolio Value: $287,232.87

Expected behavior:

- Buys more during crashes or corrections
- May outperform Fixed SIP in volatile or declining markets
- May underperform in strong bull markets if it waits too long for dips
- Sensitive to parameter choices

3. Golden Cross Strategy

The Golden Cross strategy is a trend-following strategy based on moving averages. It uses a 50-day moving average and a 200-day moving average. The 50-day moving average represents the shorter-term trend, while the 200-day moving average represents the longer-term trend.

A buy signal occurs when the 50-day moving average crosses above the 200-day moving average. This is called a golden cross and suggests that the asset may be entering an upward trend. A sell signal occurs when the 50-day moving average crosses below the 200-day moving average. This is called a death cross and suggests that the asset may be weakening.

This strategy uses two CircularQueue objects to store the most recent 50 and 200 closing prices. Each day, the strategy adds the newest close price to both queues and calculates the moving averages once enough data exists.

Parameters:

- 50-day moving average window
- 200-day moving average window
- Start year
- End year

Expected behavior:

- Avoids some large downtrends by moving to cash
- May perform well during long, clear trends
- May perform poorly in choppy markets because of late signals
- Usually has fewer trades than monthly SIP strategies

4. Momentum Strategy

The Momentum strategy invests when the asset has positive recent performance. In this project, it calculates a six-month trailing return. If the trailing return is above a selected threshold, the strategy buys or stays invested. If the trailing return falls below zero, the strategy sells or stays in cash.

This strategy is based on the idea that assets with recent positive momentum may continue rising. The reverse iterator from PriceHistory can be used to walk backward through the price list and find the price from approximately six months earlier. Since the dataset only contains trading days, the program must use available trading records rather than assuming exact calendar-day spacing.

Parameters:

- Momentum threshold
- Six-month lookback period
- Monthly rebalance schedule

- Start year
- End year

Expected behavior:

- Performs well during long upward trends
- Reduces exposure during extended downtrends
- Can miss early recoveries because it waits for momentum confirmation
- May underperform buy-and-hold or SIP strategies if signals lag too much

AI Usage Log:

Prompt:

Given the trading strategies that were implemented earlier, please give me a write up on how these trading strategies function, what parameters they used, and what the expected outcome was.

AI Output:

The AI output a detailed summary of the trading strategy implementations.

Modification:

Edited for grammar and structure.

Results and Analysis:

The following strategies were tested on the S&P 500 dataset over the required period from 2000 through 2020. The same monthly capital, start year, and end year were used for each strategy so the comparison would be fair. The main metrics used were final portfolio value, total invested, total return, compound annual growth rate, maximum drawdown, and total number of trades.

SPX Strategy Comparison

Strategy	Final Value	CAGR	Max Drawdown	Total Trades
Fixed SIP*	$276,886.13	4.06%	48.72%	240
Dynamic SIP*	$287,232.87	4.24%	45.20%	175
Golden Cross	$251,898.38	3.59%	16.05%	17
6-Month Momentum	$233,042.46	3.21%	19.21%	120

*Total invested for Fixed and Dynamic was $120,000 each

Fixed SIP vs Dynamic SIP

The Fixed Systematic Investment Plan (SIP) served as the baseline for this simulation. Its primary advantage is consistency—it requires no predictive models or technical indicators, guaranteeing steady capital deployment across the 20-year testing period regardless of market sentiment.

The Regime-Aware Dynamic SIP fundamentally improved upon this by introducing macro-trend filters and mean-reversion triggers to optimize capital efficiency. Based on the backtesting results on the S&P 500 (SPX), the Dynamic SIP significantly outperformed the Fixed SIP baseline. The final portfolio value for the Dynamic SIP reached **$287,232.87**, compared to **$276,886.13** for the Fixed SIP. This represents an absolute outperformance of **$10,346.74**, achieved without increasing the strict $120,000 budget constraint.

Early iterations of the dynamic strategy revealed that naively "investing more on dips" is often a trap. If an algorithm aggressively buys a minor 5% or 10% dip during a sustained macro crash (such as the 2008 Financial Crisis), it acts on a "falling knife" and depletes its cash reserves prematurely. Conversely, if the market rises steadily, holding too much cash in anticipation of a dip introduces "cash drag," where uninvested funds dilute overall returns.

Our research concludes that a Dynamic SIP can outperform a Fixed SIP, but only if it avoids premature deployment during sustained bear markets. By incorporating a 200-day moving average filter, our strategy successfully hoarded capital to protect against the 2000 and 2008 crashes. Through parameter sweeping, we found that the optimal trigger to deploy that hoarded capital was not a standard 10% dip, but an extreme panic threshold of 32.5% below the macro trend. Under these precise conditions, the Dynamic SIP successfully bought the generational bottom at a 70% monthly deployment rate, maximizing share accumulation at historically low prices.

Ultimately, while the Regime-Aware Dynamic SIP proved highly effective on the SPX index, its success is deeply tied to its tuned parameters. Applying these exact thresholds to a high-beta stock like NVDA would likely trigger the panic override too frequently due to the asset's inherent volatility. Therefore, a truly robust algorithmic strategy must dynamically tune its regime and panic thresholds to the specific historical volatility of the underlying asset.

Golden Cross Analysis

The Golden Cross strategy behaved differently from the SIP strategies because it was not always invested. Rather, it tried to identify broad upward or downward trends using the 50-day and 200-day moving averages. This helped the strategy avoid some periods of weakness, but it also caused delayed entries and exits.

The final value for Golden Cross was **$251,898.38** with a CAGR of **3.59%**. Its maximum drawdown was **16.05%**, which is lower than Fixed SIP. This suggests that the moving average strategy successfully reduced exposure during major declines.

Momentum Strategy Analysis

The Momentum strategy used a six-month trailing return to decide whether to be invested. It performed best when trends continued for long periods. During strong bull markets, it could remain invested and capture gains. During extended downturns, it could move to cash after momentum turned negative.

The final value for Momentum was **$233,042.46**, and its CAGR was **3.21%**. Its total number of trades was **120**. Compared with the Golden Cross strategy, Momentum was more sensitive because it rechecked performance monthly rather than relying only on moving average crossovers.

Momentum's main weakness is that it can react late. By the time the six-month return becomes negative, the asset may have already fallen significantly. Similarly, when a recovery begins, the strategy may wait until enough positive momentum appears before buying back in.

Volatile Stock Comparison (BST): NVDA or AMZN

The volatile stock comparison from the BST output showed that AMZN and NVDA had much wider return swings than SPX. For example, AMZN showed a -82.59% return in 2000 but also a 119.07% return in 2015. NVDA showed strong positive return values of 63.74% in 2015 and 89.69% in 2017. This suggests that individual stocks can move much more dramatically than a broad index like SPX.

Compared with SPX, AMZN and NVDA showed that strategy performance depends strongly on the asset being tested. A broad index is more diversified, while a single stock can experience much larger gains and losses. Therefore, the best strategy for SPX is not automatically the best strategy for every asset.

To test whether the strategies behaved differently on more volatile assets, I also ran the simulation on AMZN and NVDA and compared the results to SPX. This comparison is important because a strategy that works on a broad index like SPX may behave very differently on individual high-growth stocks.

Asset	Strategy	Final Value	CAGR	Max Drawdown	Total Trades
SPX	Fixed SIP	$276,886.13	4.06%	48.72%	240
SPX	Dynamic SIP	$287,232.87	4.24%	45.20%	175
AMZN	Fixed SIP	$4,026,430.27	18.21%	63.62%	240
AMZN	Dynamic SIP	$4,195,575.88	18.44%	63.24%	176

NVDA	Fixed SIP	$2,171,461.98	14.78%	84.60%	240
NVDA	Dynamic SIP	$2,190,338.10	14.83%	84.02%	184

The volatile stocks produced higher returns than SPX, but they also had higher maximum drawdowns. This shows the tradeoff between growth and risk. A strategy such as Dynamic SIP may benefit more from volatility because large dips create better buying opportunities. However, volatility also increases the risk of buying during a decline that continues for a long time.

Compared with SPX, AMZN and NVDA showed that strategy performance depends strongly on the asset being tested. A broad index is more diversified, while a single stock can experience much larger gains and losses. Therefore, the best strategy for SPX is not automatically the best strategy for every asset. However, in this test, Dynamic SIP was the stronger SIP-based strategy across SPX, AMZN, and NVDA because it produced a higher final value and slightly lower maximum drawdown for each asset.

Portfolio Summary:

Ticker	Shares	Average Cost	Current Price	Unrealized Return
AMZN	20	$110.00	$120.00	9.09%
NVDA	19	$111.05	$90.00	-18.96%

Portfolio Metric	Value
Cash Balance	$5,690.00
Total Market Value	$4,110.00

Total Portfolio Value	$9,800.00
Total Unrealized Return	-4.64%

AI Usage Log:
Prompt:
1. Create tables that can be used to summarize output results. Also provide a general summary I can follow of each of our trading strategy results. How do I interpret the outputs?

2. I am giving you my output screenshots, please put the values in a table to show the changes and also provide me with a conceptual summary that I can follow and write into my report.

AI Output:
Tables that are used for output information organization. Also provided a general summary that I used to interpret results conceptually. Also provided tables with our output values and gave us a summary of the outputs.

Modification:
Modifications for grammar and values checked.

Conclusion:

In conclusion, this project allowed us to explore and demonstrate how data structures can be used together in a real world financial simulation that takes into account the real world market environment. Each component of our main program had a task: the linked list stored historical price records in chronological order, iterators allowed the program to move forward and backward through the data, the circular queue supported moving average calculations, the stack allowed undo operations for trades, the queue stored pending orders in submission order, the BST organized financial results by numerical performance, and the template based stock manager allowed the program to manage different asset types using shared logic.

The strategy comparison showed that no single investment strategy is always best under every condition, but each strategy has its own benefits. We concluded that fixed SIP's advantages are that it is simple and reliable because it invests consistently. In comparison, dynamic SIP has the advantage of outperforming when the market has meaningful dips followed by recoveries, but it depends heavily on the selected parameters. Golden Cross and Momentum strategies can reduce exposure during downtrends, but they may also react late or miss parts of a recovery.

One limitation of the project is that the simulation heavily abstracts real investing practices. It does not accurately account for taxes, bid-ask spreads, slippage, transaction fees, dividend reinvestment, or emotional decision-making, etc. Another limitation is that the Dynamic SIP strategy depends on thresholds that were chosen manually. In the future, we would improve the project by adding parameter optimization, better risk metrics, dividend support, and more flexible date handling. We would also add stronger input validation and more detailed output files so results could be saved and reviewed more easily.

Overall, StockSim helped connect abstract data structure concepts to a real application. The project made it clearer why different structures exist and how choosing the correct structure affects program design, performance, and readability.

AI Usage Log:
Prompt: What should I include as a possible limitation of this project?
Response: AI gave me a list of limitations of our project.